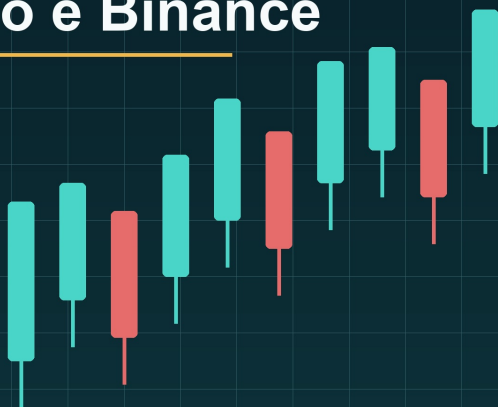


GUIA PRÁTICO

Robôs de Trade com Go e Binance



Um guia prático, simples e educativo
para criar seu primeiro bot na Spot
Testnet

[Nome do Autor]

Spot Testnet • Go 1.26 • KDP

Robôs de Trade com Go e Binance para Iniciantes

Um guia prático, simples e educativo para criar seu primeiro bot na Spot Testnet

Autor: [Nome do Autor]

Versão: 1.0

Idioma: Português do Brasil

* * *

Aviso importante

Este livro é educativo. Ele não é recomendação financeira, não é consultoria de investimento e não promete lucro.

Robôs de trade podem perder dinheiro, podem falhar por erro de código, podem sofrer com instabilidade de rede, podem executar ordens indesejadas e podem reagir mal a movimentos rápidos de mercado.

Por isso, o projeto deste livro usa a **Binance Spot Testnet**, um ambiente de testes. A ideia é aprender o processo: instalar Go, conversar com uma API, buscar preços, calcular um sinal simples e validar uma ordem sem operar dinheiro real.

Se um dia você quiser transformar um experimento em algo real, pare antes. Refaça a arquitetura, revise segurança,

limites de risco, logs, monitoramento, backtests, impostos, regras locais e termos de uso da corretora. O objetivo aqui é aprender com calma.

* * *

Como usar este livro

Este livro foi escrito para quem está começando. Você não precisa ser especialista em programação, mercado financeiro ou Go. O caminho é de projeto: primeiro entendemos as peças, depois montamos um robô pequeno, depois testamos.

Você vai encontrar explicações curtas, exemplos de terminal e trechos de código. O projeto completo acompanha o livro na pasta `codigo-robo-go-binance`. Quando um trecho aparecer aqui, ele representa a ideia central; quando quiser rodar tudo, use os arquivos do projeto.

O fluxo sugerido é:

- 1 Leia um capítulo.
- 2 Rode o comando correspondente.
- 3 Veja o erro, se houver.
- 4 Volte ao capítulo e corrija com calma.

Programar é muito menos mágico quando aceitamos que erro de terminal é parte do diálogo.

* * *

Sumário

- Capítulo 1: O que vamos construir
- Capítulo 2: O kit básico: Go, terminal e módulos
- Capítulo 3: Binance, API e Spot Testnet
- Capítulo 4: A estrutura do projeto
- Capítulo 5: Configuração segura com variáveis de ambiente
- Capítulo 6: Primeiro contato com a API: preço atual
- Capítulo 7: Velas, fechamentos e dados de mercado
- Capítulo 8: Estratégia simples com médias móveis
- Capítulo 9: Transformando a estratégia em comandos
- Capítulo 10: Ordem de teste e ordem na Testnet
- Capítulo 11: Testes automatizados
- Capítulo 12: Erros comuns e como pensar sobre eles
- Capítulo 13: Checklist de segurança
- Capítulo 14: Publicando este livro no KDP
- Apêndice A: Comandos úteis
- Apêndice B: Roteiro prático de 7 dias

- Apêndice C: Glossário do iniciante
- Apêndice D: Perguntas frequentes
- Apêndice E: Referências

* * *

Capítulo 1: O que vamos construir

Um robô de trade parece algo enorme quando a gente olha de fora. A palavra "robô" dá a impressão de inteligência, autonomia e decisões sofisticadas. Na prática, um robô simples é apenas um programa que repete um processo:

- 1 Busca dados de mercado.
- 2 Calcula alguma regra.
- 3 Decide se existe um sinal.
- 4 Registra o que aconteceu.
- 5 Opcionalmente envia uma ordem.

Neste livro, vamos construir um robô educativo com Go usando a Binance Spot Testnet. Ele será capaz de:

- consultar o preço atual de um par, como BNBUSDT;
- buscar velas de 1 minuto;
- calcular médias móveis simples;
- mostrar um sinal BUY, SELL ou HOLD;
- validar uma ordem com OrderTest;
- executar uma ordem apenas na Spot Testnet e apenas com confirmação explícita.

O objetivo não é criar uma estratégia lucrativa. O objetivo é entender o esqueleto. Depois que você entende o esqueleto, pode trocar peças: outra estratégia, outro controle de risco, outra interface, outro banco de dados, outro jeito de fazer logs.

O robô mínimo

Nosso robô será um programa de linha de comando. Isso significa que ele roda no terminal, sem tela bonita. Essa escolha é intencional. Interface gráfica pode distrair. Para aprender API, configuração e estratégia, o terminal é perfeito.

Os comandos principais serão:

```
go run ./cmd/robot price
go run ./cmd/robot klines
go run ./cmd/robot signal
go run ./cmd/robot order-test BUY
go run ./cmd/robot testnet-order BUY
```

O comando price busca preço. O comando klines mostra fechamentos recentes. O comando signal calcula a estratégia. O comando order-test valida uma ordem. O comando testnet-order só executa se você ligar uma trava de segurança.

Por que Go?

Go, ou Golang, é uma linguagem criada para ser simples, rápida de compilar e boa para programas de rede. Um robô de trade conversa com APIs pela internet, lida com respostas JSON, precisa de tempo limite, logs e testes. Go encaixa bem nesse tipo de projeto.

Neste livro usaremos Go 1.26.x. A página oficial de histórico de versões do Go lista o Go 1.26 como a linha estável lançada em 2026, com revisões menores de segurança e correção. Na prática, instale sempre o patch mais recente da linha 1.26 quando for reproduzir o projeto.

Por que Binance Spot Testnet?

A Binance oferece uma Spot Testnet para desenvolvimento. Ela simula o uso de chaves, endpoints e ordens sem usar dinheiro real. Isso permite aprender sem transformar cada erro em prejuízo.

Mesmo na Testnet, trate as chaves com respeito. Nunca coloque segredo dentro do código. Nunca publique .env em repositório. Nunca envie print de chave para alguém. O hábito que você cria no teste é o hábito que vai aparecer quando o projeto crescer.

O que este livro não cobre

Este livro não cobre futuros, margem, alavancagem, grid bot, arbitragem, backtesting profissional, banco de dados, execução contínua 24/7, deploy em servidor, Docker, monitoramento, impostos ou gestão avançada de risco.

Esses assuntos são importantes, mas não são bons para um primeiro livro de teste de publicação. Aqui a meta é completar uma jornada curta: livro simples, código funcional, pacote para KDP.

* * *

Capítulo 2: O kit básico: Go, terminal e módulos

Antes de falar com a Binance, precisamos de um ambiente de desenvolvimento. Ambiente é apenas o conjunto de ferramentas que permite escrever, rodar e testar código.

Você vai precisar de:

- Go 1.26.x;
- um terminal;
- um editor de texto;
- internet;
- uma conta na Binance Spot Testnet.

Conferindo a versão do Go

Depois de instalar Go, abra o terminal e rode:

```
go version
```

Uma resposta aceitável é algo como:

```
go version go1.26.1 darwin/arm64
```

O número exato pode mudar. O importante é estar na linha go1.26.x ou mais nova, sabendo que este projeto foi planejado para Go 1.26.

O que é um módulo Go

Um módulo é a forma moderna de organizar dependências em Go. Ele diz:

- qual é o nome do projeto;
- qual versão de Go o projeto usa;
- quais bibliotecas externas são necessárias.

No projeto deste livro, o arquivo `go.mod` começa assim:

```
module github.com/exemplo/robo-go-binance

go 1.26

require github.com/binance/binance-connector-
go/clients/spot v1.8.0
```

O nome `github.com/exemplo/robo-go-binance` é apenas um identificador. Você pode trocar se publicar seu projeto. A dependência importante é o conector oficial da Binance para Spot em Go.

Instalando dependências

Dentro da pasta do projeto, rode:

```
go mod tidy
```

Esse comando baixa as bibliotecas necessárias e cria ou atualiza o arquivo `go.sum`, que guarda checksums das dependências. Se você nunca viu um `go.sum`, não precisa decorar o conteúdo. Pense nele como um recibo técnico do que foi baixado.

Rodando testes

Antes de rodar qualquer API, rode:

```
go test ./...
```

O ./... significa "todos os pacotes abaixo desta pasta". Isso é útil porque o projeto tem várias partes: configuração, estratégia, integração com Binance e comando principal.

Se os testes passarem, você verá algo parecido com:

```
ok  github.com/exemplo/robo-go-  
      binance/internal/config  
ok  github.com/exemplo/robo-go-  
      binance/internal/strategy
```

Quando um teste falha, leia de cima para baixo. A mensagem costuma dizer o pacote, o teste e a diferença entre o valor esperado e o valor encontrado.

Terminal sem medo

O terminal é só uma conversa por texto com o computador. Você digita um comando, ele responde. Alguns comandos mostram resultado. Outros não mostram nada quando dão certo.

Três comandos úteis:

```
pwd  
ls  
go test ./...
```

pwd mostra onde você está. ls lista arquivos. go test ./... testa o projeto.

Quando algo não funcionar, volte para esses três. Muitas falhas acontecem porque estamos na pasta errada.

* * *

Capítulo 3: Binance, API e Spot Testnet

Uma API é uma porta organizada para outro sistema. Em vez de abrir o site da Binance e clicar em botões, seu programa envia requisições para endpoints. A Binance responde com dados, normalmente em JSON.

Neste livro usamos a API Spot. Spot é compra e venda direta de ativos, sem alavancagem. Mesmo assim, usaremos a Testnet.

Produção e Testnet

Ambiente de produção é onde dinheiro real mora. Ambiente de Testnet é onde aprendemos e simulamos.

O código deste livro usa esta base:

```
common.SpotRestApiTestnetUrl
```

Isso aponta para a Spot Testnet. O projeto não inclui endpoint de produção no código principal. Essa decisão é uma trava. Para um iniciante, é melhor o programa ser limitado do que perigoso.

Chave e segredo

Para consultar dados públicos, muitas vezes você não precisa de chave. Para validar ou enviar ordem, precisa.

Normalmente você terá:

- API Key: identificador público da sua chave;

- **API Secret:** segredo usado para assinar requisições privadas.

O segredo não deve ser escrito no código. O jeito simples de lidar com isso é usar variáveis de ambiente.

Exemplo:

```
export BINANCE_API_KEY="sua_chave"  
export BINANCE_API_SECRET="seu_segredo"
```

Se você fechar o terminal, talvez precise exportar de novo. Isso é normal.

O conector oficial em Go

A Binance mantém um repositório chamado `binance-connector-go`. Para Spot, usamos:

```
go get github.com/binance/binance-connector-  
go/clients/spot
```

No nosso `go.mod`, fixamos a versão `v1.8.0`. Fixar versão ajuda a evitar surpresas. Se a biblioteca mudar no futuro, o livro continua reproduzível.

REST, WebSocket API e WebSocket Streams

O SDK separa três formas de conversar com a Binance:

- **REST API:** chamadas pontuais, como "qual é o preço agora?";
- **WebSocket API:** chamadas por conexão WebSocket;
- **WebSocket Streams:** fluxo de eventos em tempo real.

Neste primeiro projeto, vamos usar REST. É mais simples de explicar, mais fácil de testar e suficiente para aprender.

Limites e tempo

APIs têm limites. Se você pedir dados demais, pode receber erro. Se a rede estiver lenta, pode passar do tempo. Por isso, nosso código usa `context.WithTimeout`, uma ferramenta do Go para dizer: "tente, mas não fique preso para sempre".

Trecho do comando principal:

```
ctx, cancel :=  
context.WithTimeout(context.Background(), cfg.Timeout)  
defer cancel()
```

Esse padrão aparece muito em programas Go que falam com serviços externos.

* * *

Capítulo 4: A estrutura do projeto

O projeto completo está na pasta `codigo-robo-go-binance`. Ele foi organizado em pacotes pequenos:

```
codigo-robo-go-binance/  
  cmd/robot/main.go  
  internal/binance/client.go  
  internal/config/config.go  
  internal/runner/runner.go  
  internal/strategy/sma.go  
  go.mod  
  .env.example  
  README.md
```

Parece muita coisa, mas a divisão é simples.

cmd/robot

Aqui fica o ponto de entrada. É o arquivo que recebe o comando do terminal:

```
go run ./cmd/robot signal
```

Ele carrega configuração, cria um contexto com timeout, monta o Runner e chama a ação certa.

internal/config

Aqui fica a leitura de variáveis de ambiente. Em vez de espalhar os `.Getenv` pelo projeto todo, concentramos tudo em um lugar.

Isso facilita teste e reduz bagunça.

internal/binance

Aqui fica o cliente que conversa com a Binance. Ele sabe buscar preço, buscar fechamentos e enviar chamadas de ordem na Testnet.

É importante separar essa parte porque API externa muda mais do que cálculo interno. Se amanhã quisermos trocar SDK ou adicionar logs, mexemos aqui.

internal/strategy

Aqui fica a lógica da estratégia. Essa parte não precisa saber o que é Binance. Ela recebe uma lista de preços e devolve um sinal.

Essa separação é ouro para testes. Você consegue testar a estratégia sem internet, sem chave e sem Testnet.

internal/runner

Aqui fica a cola. O Runner junta configuração, cliente Binance e estratégia para transformar tudo em comandos úteis.

Por que usar internal?

Em Go, a pasta internal tem um significado especial: ela indica código interno do projeto. Outros módulos não devem importar esses pacotes diretamente.

Para um projeto pequeno, isso pode parecer formalidade. Mas é um bom hábito. Ele separa o comando público do miolo interno.

* * *

Capítulo 5: Configuração segura com variáveis de ambiente

Um erro comum de iniciante é colocar chave dentro do código:

```
apiKey := "minha-chave-aqui"
```

Não faça isso. Mesmo em Testnet, evite. Código costuma ser copiado, enviado, publicado e esquecido. Segredo dentro de código vaza.

O projeto usa variáveis de ambiente:

```
BINANCE_API_KEY  
BINANCE_API_SECRET  
BOT_SYMBOL  
BOT_QUANTITY  
BOT_SHORT_WINDOW  
BOT_LONG_WINDOW  
BOT_ALLOW_TESTNET_ORDER
```

O arquivo .env.example

O projeto inclui um exemplo:

```
BINANCE_API_KEY=coloque_sua_chave_da_spot_testnet_aqui  
BINANCE_API_SECRET=coloque_seu_segredo_da_spot_testnet_aqui  
  
BOT_SYMBOL=BNBUSDT  
BOT_QUANTITY=0.01  
BOT_SHORT_WINDOW=7  
BOT_LONG_WINDOW=25  
BOT_ALLOW_TESTNET_ORDER=false
```

Esse arquivo pode ser publicado porque não tem segredo real. O arquivo .env, com seus valores de verdade, não deve

ser publicado.

Carregando configuração

O pacote config cria uma estrutura:

```
type Config struct {
    APIKey          string
    APISecret       string
    Symbol          string
    Quantity        float32
    ShortWindow     int
    LongWindow      int
    AllowTestnetOrder bool
    Timeout         time.Duration
}
```

Essa estrutura vira o retrato da configuração do robô.

Defaults úteis

O código usa valores padrão:

```
cfg := Config{
    Symbol:      "BNBUSDT",
    Quantity:    0.01,
    ShortWindow: 7,
    LongWindow:  25,
    Timeout:     10 * time.Second,
}
```

Isso permite rodar comandos públicos sem configurar tudo.

Para ordens, porém, as chaves continuam obrigatórias.

A trava mais importante

A variável mais importante é:

```
BOT_ALLOW_TESTNET_ORDER=false
```

Enquanto ela estiver falsa, o comando testnet-order não envia ordem. Para liberar, você precisa definir:

```
export BOT_ALLOW_TESTNET_ORDER=true
```

Essa trava parece chata, mas é boa. Robôs devem exigir intenção clara para executar ação sensível.

Validação simples

O código valida se a janela curta é menor que a longa:

```
if cfg.ShortWindow >= cfg.LongWindow {  
    return Config{}, errors.New(  
        "BOT_SHORT_WINDOW deve ser menor que  
        BOT_LONG_WINDOW",  
    )  
}
```

Essa regra evita uma estratégia sem sentido. Se a média curta tiver o mesmo tamanho da média longa, não existe cruzamento útil entre elas.

* * *

Capítulo 6: Primeiro contato com a API: preço atual

Agora vamos buscar preço. Esse é o primeiro comando que conversa com a Binance:

```
go run ./cmd/robot price
```

Se tudo estiver certo, a saída será parecida com:

```
BNBUSDT price: 587.12345678
```

O número muda o tempo todo. Isso é esperado.

Criando o cliente

O cliente é criado com a base da Spot Testnet:

```
restCfg := common.NewConfigurationRestAPI(  
    common.WithBasePath(common.SpotRestApiTestnetUrl),  
    common.WithTimeout(cfg.Timeout),  
    common.WithRetries(2),  
)
```

Observe que não usamos URL de produção. Essa escolha aparece no código, não apenas no texto.

Depois criamos o cliente Spot:

```
api := spot.NewBinanceSpotClient(  
    spot.WithRestAPI(restCfg),  
)
```

Buscando preço

O método Price faz a chamada:

```
resp, err := c.api.RestApi.MarketAPI.  
    TickerPrice(ctx).  
    Symbol(symbol).  
    Execute()
```

Esse estilo encadeado é comum em SDKs gerados a partir de especificações de API. Você começa com o grupo da API, escolhe o endpoint, define parâmetros e executa.

Convertendo string para número

A Binance devolve preço como string. Exemplo:

```
{
  "symbol": "BNBUSDT",
  "price": "587.12345678"
}
```

Para calcular médias, precisamos converter:

```
price, err := strconv.ParseFloat(
    resp.Data.TickerPriceResponse1.GetPrice(),
    64,
)
```

Dinheiro e preço exigem cuidado com precisão. Em sistemas profissionais, você vai estudar decimal fixo, casas mínimas, filtros de símbolo e arredondamento. Neste projeto educativo, float64 é suficiente para aprender a estrutura.

Se der erro

Erros comuns neste comando:

- sem internet;
- símbolo inválido;
- Testnet indisponível;
- mudança temporária na API;
- limite de requisições.

Leia a mensagem. Se parecer erro de rede, tente de novo alguns minutos depois. Se parecer símbolo inválido, confira BOT_SYMBOL.

* * *

Capítulo 7: Velas, fechamentos e dados de mercado

Preço atual é útil, mas uma estratégia precisa de histórico. Para médias móveis, precisamos de uma sequência de fechamentos.

Uma vela, ou candlestick, resume um intervalo de tempo. No nosso caso, usamos velas de 1 minuto.

Cada vela costuma ter:

- abertura;
- máxima;
- mínima;
- fechamento;
- volume;
- horário.

Para médias móveis, vamos usar apenas o fechamento.

O comando klines

Rode:

```
go run ./cmd/robot klines
```

Você verá uma lista de fechamentos recentes:

```
BNBUSDT ultimos fechamentos de 1m:  
01 587.10000000  
02 587.20000000  
03 587.05000000
```

O projeto busca longWindow + 1 fechamentos. Se a janela longa é 25, buscamos 26. O motivo aparece no próximo capítulo: para detectar cruzamento, precisamos comparar o estado anterior com o estado atual.

Buscando Klines

O método usa o endpoint de Klines:

```
resp, err := c.api.RestApi.MarketAPI.Klines(ctx).
    Symbol(symbol).
    Interval(models.KlinesIntervalParameterInterval1m).
    Limit(limit).
    Execute()
```

O intervalo está fixado em 1 minuto para simplificar. Você poderia transformar isso em variável depois.

Pegando o fechamento

Na resposta de Klines, o fechamento fica na posição 4 da vela. O código extrai assim:

```
for _, candle := range resp.Data.Items {
    closePrice, err := strconv.ParseFloat(
        *candle.Items[4].String,
        64,
    )
    closes = append(closes, closePrice)
}
```

Essa parte é propositalmente defensiva no projeto completo: ela verifica se a vela tem itens suficientes e se o fechamento existe.

Dados ruins entram, sinais ruins saem

Robôs dependem de dados. Se a API devolve algo inesperado, se você interpreta a posição errada ou se

converte número errado, a estratégia vai tomar decisões ruins.

Por isso, uma regra simples:

Nunca confie cegamente no formato de uma resposta externa.

Valide. Trate erro. Faça logs. Em projeto de estudo, isso parece exagero. Em projeto real, isso é sobrevivência.

* * *

Capítulo 8: Estratégia simples com médias móveis

Agora chegamos à parte que parece "inteligência" do robô. Na verdade, será uma regra simples.

Vamos usar duas médias móveis simples:

- SMA curta: reage mais rápido;
- SMA longa: reage mais devagar.

Quando a média curta cruza para cima da longa, interpretamos como BUY. Quando cruza para baixo, interpretamos como SELL. Quando não há cruzamento, HOLD.

O que é SMA?

SMA significa Simple Moving Average, ou média móvel simples.

Se os preços são:

10, 20, 30

A média é:

$(10 + 20 + 30) / 3 = 20$

Em Go:

```
func SMA(values []float64, window int) (float64,
error) {
    if len(values) < window {
        return 0, fmt.Errorf("dados insuficientes")
    }
}
```

```

        start := len(values) - window
        var sum float64
        for _, value := range values[start:] {
            sum += value
        }
        return sum / float64(window), nil
    }
}

```

O projeto completo tem validações extras, mas a ideia é essa.

Por que comparar antes e agora?

Se a SMA curta está acima da longa agora, isso não significa necessariamente compra. Talvez ela já estivesse acima há muito tempo.

Para detectar cruzamento, comparamos:

- média curta anterior;
- média longa anterior;
- média curta atual;
- média longa atual.

Regra de compra:

```

antes: curta <= longa
agora: curta > longa

```

Regra de venda:

```

antes: curta >= longa
agora: curta < longa

```

A decisão

O projeto representa a decisão assim:

```
type Decision struct {  
    Signal    Signal  
    ShortSMA  float64  
    LongSMA   float64  
    LastClose float64  
}
```

O robô não devolve apenas BUY ou SELL. Ele também mostra os números que levaram ao sinal. Isso é útil para aprender e depurar.

O comando signal

Rode:

```
go run ./cmd/robot signal
```

Exemplo de saída:

```
symbol: BNBUSD  
last_close: 587.10000000  
short_sma_7: 586.92000000  
long_sma_25: 586.70000000  
signal: HOLD
```

Na maioria das vezes, o sinal será HOLD. Isso é normal. Cruzamentos não acontecem a cada execução.

Estratégia simples não é estratégia boa

Médias móveis são fáceis de explicar, mas sofrem com atraso. Elas olham para trás. Em mercado lateral, podem gerar sinais ruins. Em movimentos rápidos, podem entrar tarde.

Isso não invalida o aprendizado. A primeira estratégia deve ser compreensível. Antes de buscar sofisticação, você precisa aprender a montar o ciclo completo.

* * *

Capítulo 9: Transformando a estratégia em comandos

Até aqui temos peças:

- configuração;
- cliente Binance;
- busca de preço;
- busca de velas;
- cálculo de sinal.

Agora juntamos tudo em comandos.

O papel do Runner

O pacote runner é a cola. Ele recebe a configuração, cria o cliente e expõe métodos como:

```
func (r Runner) Price(ctx context.Context) error
func (r Runner) Klines(ctx context.Context) error
func (r Runner) Signal(ctx context.Context) error
```

Isso deixa o main.go pequeno.

O main.go

O comando principal lê argumentos:

```
switch args[0] {
case "price":
    return app.Price(ctx)
case "klines":
    return app.Klines(ctx)
case "signal":
    return app.Signal(ctx)
```

```
case "order-test":
    return app.OrderTest(ctx, argOrDefault(args, 1,
"BUY"))
case "testnet-order":
    return app.TestnetOrder(ctx, argOrDefault(args, 1,
"BUY"))
}
```

Não usamos biblioteca de CLI externa. Para um primeiro projeto, a biblioteca padrão basta.

Comandos pequenos são bons

Cada comando faz uma coisa:

- price: "a API responde?";
- klines: "consigo pegar histórico?";
- signal: "minha estratégia calcula?";
- order-test: "minhas chaves e parâmetros validam?";
- testnet-order: "consigo executar na Testnet com trava ligada?".

Essa separação ajuda a diagnosticar falhas. Se price falha, o problema é básico: rede, símbolo ou API. Se price funciona e signal falha, olhe Klines ou estratégia. Se signal funciona e order-test falha, olhe chaves e permissões.

Side: BUY ou SELL

O comando de ordem aceita lado:

```
go run ./cmd/robot order-test BUY
go run ./cmd/robot order-test SELL
```

O projeto também aceita compra e venda, mas no livro vamos preferir BUY e SELL, porque são os termos usados pela API.

Por que não rodar em loop?

Um robô real costuma rodar continuamente. Ele acorda a cada intervalo, busca dados, calcula sinal e age. Neste livro, escolhemos execução única.

Motivos:

- é mais fácil entender;
- é mais fácil testar;
- reduz risco de ordem repetida;
- evita precisar falar de serviço 24/7;
- combina com um livro curto para iniciantes.

Depois que você entender a execução única, transformar em loop é um próximo projeto.

* * *

Capítulo 10: Ordem de teste e ordem na Testnet

Agora entramos na parte sensível: ordens.

Mesmo na Testnet, vamos separar duas coisas:

- `order-test`: valida a ordem, mas não executa;
- `testnet-order`: executa uma ordem na Testnet, se a trava estiver ligada.

Primeiro: configure as chaves

Para comandos de ordem, exporte:

```
export BINANCE_API_KEY="sua_chave_da_spot_testnet"
export
BINANCE_API_SECRET="seu_segredo_da_spot_testnet"
```

Use chaves da Spot Testnet, não de produção.

Rodando `order-test`

O comando:

```
go run ./cmd/robot order-test BUY
```

Ele chama o endpoint de teste:

```
_, err := c.api.RestApi.TradeAPI.OrderTest(ctx).
    Symbol(cfg.Symbol).
    Side(side).
    Type(models.NewOrderTypeParameterMarket).
    Quantity(cfg.Quantity).
    Execute()
```

Se der certo, a ordem é considerada válida, mas não é enviada ao livro de ofertas.

Ordem real na Testnet

O comando de execução é:

```
go run ./cmd/robot testnet-order BUY
```

Mas ele só passa se você ligar:

```
export BOT_ALLOW_TESTNET_ORDER=true
```

Sem isso, o projeto devolve erro:

```
ordem bloqueada: defina BOT_ALLOW_TESTNET_ORDER=true  
apenas na Spot Testnet
```

Essa é uma trava simples e clara. Em um projeto maior, você teria outras:

- limite máximo por ordem;
- limite diário;
- bloqueio por símbolo;
- confirmação manual;
- dry run;
- kill switch.

Mercado, quantidade e filtros

O projeto usa ordem de mercado:

```
Type(models.NewOrderTypeParameterMarket)
```

E quantidade:

```
Quantity(cfg.Quantity)
```

Na Binance, cada símbolo tem regras: quantidade mínima, passo de quantidade, notional mínimo e outros filtros. Este livro não mergulha nesses detalhes. Se uma ordem for rejeitada por filtro, ajuste BOT_QUANTITY e consulte as regras do símbolo.

Não automatize dinheiro real a partir deste livro

Este capítulo ensina mecânica de API. Não ensina uma operação lucrativa. O código não tem proteção profissional, não tem gestão de posição, não tem reconciliação de ordens, não tem persistência e não tem monitoramento.

Use como aprendizado. Não como promessa.

* * *

Capítulo 11: Testes automatizados

Teste é o jeito de fazer o código explicar se ainda funciona.

Neste projeto, testes importantes não dependem da Binance. Isso é intencional. Se todo teste precisasse de internet, chave e API disponível, testar seria lento e frágil.

Testando a SMA

O teste mais simples:

```
func TestSMA(t *testing.T) {
    got, err := SMA([]float64{10, 20, 30, 40}, 3)
    if err != nil {
        t.Fatalf("SMA() error = %v", err)
    }
    if got != 30 {
        t.Fatalf("SMA() = %v, want 30", got)
    }
}
```

Os últimos três valores são 20, 30 e 40. A média é 30. O teste é quase uma conta de papel.

Testando sinal de compra

Para compra, usamos dados artificiais:

```
decision, err := MovingAverageSignal(
    []float64{10, 10, 10, 10, 20},
    2,
    4,
)
```

A alta no último valor faz a média curta cruzar para cima. O teste espera BUY.

Testando a trava de ordem

Uma das partes mais importantes:

```
func TestRequireOrderPermissionBlocksByDefault(t
*testing.T) {
    cfg := Config{}
    if err := cfg.RequireOrderPermission(); err == nil
{
    t.Fatal("want blocked order error")
}
}
```

Esse teste garante que a configuração vazia não libera ordem. Segurança boa é segurança que falha fechada.

Rodando todos os testes

Use:

```
go test ./...
```

Antes de mexer em estratégia, rode testes. Depois de mexer, rode de novo. Esse ciclo evita que você conserte uma coisa quebrando outra.

Testes não provam lucro

Testes automatizados provam que o código segue regras esperadas. Eles não provam que a estratégia ganha dinheiro.

Para estudar performance, você precisaria de backtest, dados históricos confiáveis, custos, slippage, validação fora da amostra e análise de risco. Isso fica para outro livro.

* * *

Capítulo 12: Erros comuns e como pensar sobre eles

Todo projeto técnico tem erros. A diferença entre iniciante e pessoa experiente não é "não errar". É saber investigar.

"command not found: go"

O Go não está instalado ou não está no PATH. Reinstale Go ou ajuste o terminal.

Teste:

```
go version
```

"no such file or directory"

Você provavelmente está na pasta errada. Rode:

```
pwd  
ls
```

Entre na pasta do projeto:

```
cd codigo-robo-go-binance
```

"BOT_SHORT_WINDOW deve ser menor que BOT_LONG_WINDOW"

Você configurou janelas inválidas. Use, por exemplo:

```
export BOT_SHORT_WINDOW=7  
export BOT_LONG_WINDOW=25
```

"defina BINANCE_API_KEY e BINANCE_API_SECRET"

Você tentou rodar comando de ordem sem chaves.

Para preço e sinal, talvez não precise. Para order-test e testnet-order, precisa.

"ordem bloqueada"

Isso é bom. Significa que a trava funcionou.

Para executar na Testnet:

```
export BOT_ALLOW_TESTNET_ORDER=true
```

Leia de novo: Testnet. Não produção.

Erro de filtro da Binance

Algo como quantidade mínima ou passo inválido. Ajuste BOT_QUANTITY.

Exemplo:

```
export BOT_QUANTITY=0.02
```

Se continuar, consulte ExchangeInfo do símbolo na documentação da Binance.

Timeout

Rede lenta ou API demorando. Tente novamente. Se acontecer sempre, aumente timeout no código ou revise conexão.

Erro HTTP 451

O erro 451 normalmente indica que o serviço não está disponível para determinada região, jurisdição ou condição de acesso. Se isso acontecer, não trate como bug da estratégia. Trate como uma regra do ambiente.

O caminho correto é respeitar as regras locais, os termos da Binance e as opções oficiais disponíveis para a sua região. Para fins de estudo, você ainda pode ler o código, rodar testes unitários e entender a arquitetura sem enviar requisições para a API.

A estratégia só dá HOLD

Normal. Cruzamento exige mudança. Se você rodar em um momento sem cruzamento, HOLD é a resposta correta.

Não force compra só porque quer ver ação. Esse impulso é um dos maiores inimigos de quem automatiza trade.

* * *

Capítulo 13: Checklist de segurança

Antes de brincar com qualquer ordem, revise este checklist.

Chaves

- Use apenas chaves da Spot Testnet.
- Não cole chaves em prints.
- Não escreva chaves no código.
- Não publique .env.
- Revogue chaves que você suspeita que vazaram.

Código

- Confirme que o endpoint é SpotRestApiTestnetUrl.
- Rode go test ./....
- Leia o comando antes de executar.
- Comece com order-test, não com testnet-order.
- Use quantidade pequena mesmo na Testnet.

Estratégia

- Entenda o que BUY, SELL e HOLD significam.
- Não confunda sinal com garantia.
- Não aumente quantidade para "compensar" erro.

- Não opere produção com estratégia de exemplo.

Operação

- Tenha logs.
- Tenha limite de perda.
- Tenha botão de parar.
- Monitore ordens abertas.
- Compare saldo esperado e saldo real.

Publicação

Se você publicar este livro ou um derivado, revise:

- direitos de uso de nomes e marcas;
- ausência de promessa financeira;
- clareza de aviso de risco;
- disclosure de conteúdo gerado por IA quando aplicável;
- preview do EPUB no Kindle Previewer.

* * *

Capítulo 14: Publicando este livro no KDP

Este projeto também existe para testar publicação na Amazon KDP. Então vamos tratar o livro como produto editorial simples.

Formato principal

O KDP aceita formatos como DOCX, KPF e EPUB para eBooks reflowable. A própria ajuda do KDP em português informa que DOC/DOCX costuma converter bem, que KPF pode ser criado com Kindle Create e que EPUB é aceito quando segue as diretrizes Kindle.

Nosso fluxo:

- 1 Escrever em Markdown.
- 2 Gerar EPUB.
- 3 Gerar DOCX para revisão e Kindle Create.
- 4 Gerar PDF para leitura de conferência.
- 5 Fazer preview antes de publicar.

Por que Markdown?

Markdown é simples, limpo e fácil de versionar. Você escreve título com #, subtítulo com ##, listas com - e código com três crases.

Exemplo:

```
# Capítulo 1

Texto do capítulo.

~~~go
fmt.Println("ola")
~~~
```

Para um livro técnico curto, Markdown é ótimo como fonte.

EPUB

EPUB é um pacote com HTML, estilos e metadados. O Kindle consegue importar EPUB no KDP, mas ainda é obrigatório fazer preview. Código pode quebrar linha de um jeito estranho em telas pequenas.

Por isso, mantenha trechos curtos. Evite linhas longas.

DOCX e Kindle Create

O Kindle Create aceita DOC/DOCX para livros reflowable e pode exportar KPF, que a Amazon recomenda para melhor experiência em dispositivos Kindle.

O DOCX também serve para revisão humana. Algumas pessoas preferem comentar em Word, Pages ou Google Docs.

PDF

PDF não é o melhor formato para eBook reflowable, mas é ótimo para revisão visual. Você consegue olhar quebras, sumário, títulos e blocos de código.

Não confunda PDF de revisão com arquivo final ideal para Kindle.

Capa

Para este teste, a capa deve ser simples:

- título grande;
- subtítulo claro;
- visual técnico;
- sem logos oficiais da Binance ou Go;
- sem promessas de lucro.

Uma capa honesta vende melhor para o leitor certo do que uma capa agressiva com promessa impossível.

Metadados

Metadados ajudam o leitor a encontrar o livro:

- título;
- subtítulo;
- descrição;
- autor;
- palavras-chave;
- categorias;
- preço;
- território;
- declaração de IA quando aplicável.

Como este manuscrito foi produzido com ajuda de IA, o checklist recomenda marcar conteúdo AI-generated no KDP se você usar este texto ou capa como base final sem reescrita humana substancial.

Descrição curta sugerida

Aprenda, passo a passo, a criar um robô educativo de trade usando Go e a Binance Spot Testnet. Este guia para iniciantes mostra como instalar Go, configurar variáveis de ambiente, buscar preços, ler velas, calcular médias móveis, validar ordens e preparar um pequeno projeto para estudo.

Sem promessa de lucro. Sem operação com dinheiro real. Apenas um caminho prático para entender APIs, robôs e publicação de um ebook técnico simples no KDP.

Checklist de upload

Antes de publicar:

- Abra o EPUB no Kindle Previewer.
- Verifique sumário.
- Verifique blocos de código.
- Confira capa em miniatura.
- Revise aviso de risco.
- Preencha metadados.
- Marque disclosure de IA se aplicável.
- Publique como teste com preço simples.

* * *

Apêndice A: Comandos úteis

Ambiente

```
go version
go mod tidy
go test ./...
```

Configuração

```
export BINANCE_API_KEY="sua_chave"
export BINANCE_API_SECRET="seu_segredo"
export BOT_SYMBOL="BNBUSDT"
export BOT_QUANTITY="0.01"
export BOT_SHORT_WINDOW="7"
export BOT_LONG_WINDOW="25"
export BOT_ALLOW_TESTNET_ORDER="false"
```

Robô

```
go run ./cmd/robot price
go run ./cmd/robot klines
go run ./cmd/robot signal
go run ./cmd/robot order-test BUY
```

Ordem na Testnet

```
export BOT_ALLOW_TESTNET_ORDER=true
go run ./cmd/robot testnet-order BUY
```

Depois do teste:

```
export BOT_ALLOW_TESTNET_ORDER=false
```

Investigação rápida

```
pwd
ls
go env
go test ./...
```

* * *

Apêndice B: Roteiro prático de 7 dias

Este roteiro é opcional, mas ajuda se você quiser transformar a leitura em prática. A ideia é estudar pouco por dia e terminar com o projeto rodando.

Não pule direto para ordens. O valor deste livro está em entender as camadas. Quando você entende as camadas, o terminal deixa de parecer uma caixa misteriosa.

Dia 1: Ambiente e projeto

Meta do dia: conseguir rodar Go e abrir a pasta certa.

Faça:

```
go version
cd codigo-robo-go-binance
ls
go test ./...
```

Se `go version` falhar, pare. Resolva instalação antes de continuar. Se `cd` falhar, encontre a pasta do projeto. Se `go test ./...` falhar, leia o erro com calma.

Anote:

- Qual versão de Go apareceu?
- Em qual pasta o projeto está?
- Os testes passaram?

O aprendizado do dia não é "programar um robô". É aprender a preparar o chão. Projetos quebram muito menos

quando o chão está firme.

Dia 2: Configuração sem segredo no código

Meta do dia: entender variáveis de ambiente.

Abra `.env.example` e leia cada linha. Não cole chaves reais ainda. Depois rode:

```
export BOT_SYMBOL="BNBUSDT"
export BOT_QUANTITY="0.01"
export BOT_SHORT_WINDOW="7"
export BOT_LONG_WINDOW="25"
export BOT_ALLOW_TESTNET_ORDER="false"
go test ./...
```

Agora experimente um erro controlado:

```
export BOT_SHORT_WINDOW="25"
export BOT_LONG_WINDOW="7"
go run ./cmd/robot signal
```

Você deve ver erro de validação. Isso é bom. Um programa que recusa configuração ruim está protegendo você.

Depois volte ao normal:

```
export BOT_SHORT_WINDOW="7"
export BOT_LONG_WINDOW="25"
```

Anote:

- O que acontece quando a janela curta é maior que a longa?
- Por que `BOT_ALLOW_TESTNET_ORDER` começa como `false`?

- Onde suas chaves ficariam se você fosse usar ordem de teste?

Dia 3: Preço atual

Meta do dia: fazer uma chamada pública de mercado.

Rode:

```
go run ./cmd/robot price
```

Se aparecer preço, ótimo. Se falhar, investigue:

```
pwd
go test ./...
echo $BOT_SYMBOL
```

Troque o símbolo:

```
export BOT_SYMBOL="BTCUSDT"
go run ./cmd/robot price
```

Depois volte para:

```
export BOT_SYMBOL="BNBUSDT"
```

O ponto do dia é perceber que o robô não "sabe" nada sozinho. Ele depende de configuração e da resposta da API.

Anote:

- Qual símbolo você testou?
- A resposta mudou?
- O erro, se apareceu, foi de rede, símbolo ou código?

Dia 4: Velas e fechamentos

Meta do dia: buscar uma sequência de dados.

Rode:

```
go run ./cmd/robot klines
```

Leia os números. Eles são os fechamentos das velas de 1 minuto. Agora mude a janela longa:

```
export BOT_LONG_WINDOW="10"  
go run ./cmd/robot klines
```

Você deve ver menos fechamentos. O projeto busca `longWindow + 1`, porque precisa de um estado anterior e um estado atual.

Volte ao padrão:

```
export BOT_LONG_WINDOW="25"
```

Anote:

- Quantos fechamentos aparecem quando `BOT_LONG_WINDOW=10`?
- Por que precisamos de uma vela extra?
- O que aconteceria se a API devolvesse menos dados?

Dia 5: Estratégia e sinal

Meta do dia: entender BUY, SELL e HOLD.

Rode:

```
go run ./cmd/robot signal
```

Leia a saída:

```
symbol: BNBUSDT  
last_close: ...  
short_sma_7: ...  
long_sma_25: ...
```

```
signal: ...
```

O sinal mais comum será HOLD. Não tente forçar uma compra. Robô bom é robô que sabe não fazer nada.

Agora experimente janelas menores:

```
export BOT_SHORT_WINDOW="3"  
export BOT_LONG_WINDOW="8"  
go run ./cmd/robot signal
```

Janelas menores reagem mais rápido, mas também podem gerar mais ruído. Janelas maiores reagem mais devagar, mas podem filtrar movimentos pequenos.

Volte ao padrão:

```
export BOT_SHORT_WINDOW="7"  
export BOT_LONG_WINDOW="25"
```

Anote:

- O sinal mudou?
- A média curta ficou acima ou abaixo da longa?
- Você entendeu por que isso não prova lucro?

Dia 6: Ordem de teste

Meta do dia: validar uma ordem sem executar.

Crie chaves na Binance Spot Testnet e exporte:

```
export BINANCE_API_KEY="sua_chave_da_testnet"  
export BINANCE_API_SECRET="seu_segredo_da_testnet"
```

Rode:

```
go run ./cmd/robot order-test BUY
```

Se falhar por quantidade, ajuste:

```
export BOT_QUANTITY="0.02"  
go run ./cmd/robot order-test BUY
```

Se falhar por chave, confira se você está usando Testnet. Chave de produção e chave de Testnet não são a mesma coisa.

Anote:

- A ordem de teste validou?
- Qual quantidade funcionou?
- Qual erro apareceu antes de funcionar?

Dia 7: Execução na Spot Testnet e revisão

Meta do dia: executar uma ordem na Testnet com intenção explícita.

Primeiro confira:

```
echo $BOT_SYMBOL  
echo $BOT_QUANTITY  
echo $BOT_ALLOW_TESTNET_ORDER
```

Se tudo estiver certo e você quiser executar na Testnet:

```
export BOT_ALLOW_TESTNET_ORDER="true"  
go run ./cmd/robot testnet-order BUY
```

Depois desligue:

```
export BOT_ALLOW_TESTNET_ORDER="false"
```

Agora revise tudo:

```
go test ./...  
go run ./cmd/robot price  
go run ./cmd/robot signal
```

Anote:

- Você entendeu cada comando?
- Sabe explicar por que a trava existe?
- Sabe onde trocar a estratégia?
- Sabe por que isso ainda não é um robô para dinheiro real?

Se a resposta for sim, o projeto cumpriu o papel dele.

* * *

Apêndice C: Glossário do iniciante

API

API é uma forma organizada de um programa conversar com outro. No nosso caso, o programa em Go conversa com a Binance. Em vez de clicar no site, o código envia uma requisição.

Endpoint

Endpoint é um endereço específico da API. Um endpoint busca preço. Outro busca velas. Outro valida ordem. Cada endpoint tem parâmetros e regras.

REST

REST é um estilo comum de API baseado em requisições HTTP. O programa pergunta, a API responde. Para começar, REST é mais simples que fluxo em tempo real.

WebSocket

WebSocket mantém uma conexão aberta. É útil para receber dados em tempo real. Este livro não usa WebSocket no projeto principal para manter o primeiro passo simples.

Spot

Spot é o mercado de compra e venda direta de ativos. Se você compra BNB usando USDT no spot, está trocando um ativo por outro sem alavancagem.

Testnet

Testnet é ambiente de teste. Ele imita parte do comportamento de produção, mas não usa dinheiro real. É o lugar certo para aprender.

Produção

Produção é o ambiente real. Em corretora, produção significa risco real. Este livro evita produção no código principal.

API Key

API Key identifica sua aplicação para a corretora. Ela não deve ser tratada como texto público, embora o maior segredo seja a API Secret.

API Secret

API Secret assina requisições privadas. Nunca coloque esse valor no código, em print público ou em repositório.

Variável de ambiente

Variável de ambiente é uma configuração guardada no sistema ou terminal. O código lê esse valor sem precisar gravá-lo no arquivo fonte.

Símbolo

Símbolo é o par negociado, como BNBUSDT ou BTCUSDT. Ele indica o ativo base e o ativo de cotação.

Ativo base

No símbolo BNBUSDT, BNB é o ativo base. Quando você compra BNBUSDT, compra BNB usando USDT.

Ativo de cotação

No símbolo BNBUSDT, USDT é o ativo de cotação. Ele é a unidade em que o preço é expresso.

Kline

Kline é a vela de mercado. Ela contém abertura, máxima, mínima, fechamento e volume de um intervalo.

Fechamento

Fechamento é o último preço de uma vela. A estratégia deste livro usa apenas os fechamentos.

Média móvel simples

Média móvel simples, ou SMA, é a média dos últimos n valores. Se a janela é 7, calculamos a média dos últimos 7 fechamentos.

Cruzamento

Cruzamento acontece quando uma média passa a outra. Neste livro, usamos o cruzamento da média curta com a longa como sinal educativo.

BUY

BUY significa compra. No projeto, é apenas um sinal ou lado de ordem. Não significa que a operação é boa.

SELL

SELL significa venda. Pode indicar saída ou venda do ativo, dependendo do contexto. Neste projeto, é apenas um lado de ordem.

HOLD

HOLD significa não agir. Em robôs, não fazer nada é uma decisão legítima.

Ordem de mercado

Ordem de mercado tenta executar imediatamente pelo preço disponível. É simples, mas pode sofrer slippage.

Slippage

Slippage é a diferença entre o preço esperado e o preço de execução. Em mercado rápido ou sem liquidez, essa diferença pode ser relevante.

Liquidez

Liquidez é a facilidade de comprar ou vender sem mexer muito no preço. Pares com pouca liquidez podem ser mais perigosos para automação.

Filtro de símbolo

Filtro de símbolo é uma regra da corretora, como quantidade mínima ou passo de quantidade. Uma ordem pode ser rejeitada se violar filtro.

Timeout

Timeout é um limite de tempo. Se a API demora demais, o programa para de esperar e devolve erro.

Contexto em Go

`context.Context` carrega cancelamento, timeout e valores entre chamadas. Em programas de rede, é ferramenta essencial.

Módulo Go

Módulo Go é a unidade de organização de dependências. O arquivo `go.mod` declara o nome do módulo e as bibliotecas usadas.

Teste unitário

Teste unitário verifica uma parte pequena do código. A estratégia de SMA é boa para teste unitário porque não depende de API.

Dry run

Dry run é uma execução simulada. O comando `order-test` é parecido com isso: valida sem executar a ordem.

Kill switch

Kill switch é um mecanismo para parar o robô rapidamente. Este projeto não implementa um kill switch completo, mas a trava `BOT_ALLOW_TESTNET_ORDER` ensina a mentalidade.

* * *

Apêndice D: Perguntas frequentes

Posso usar este robô com dinheiro real?

Não como está. O projeto é educativo e usa Spot Testnet. Ele não tem gestão de risco profissional, persistência, monitoramento, reconciliação de ordens, tratamento avançado de filtros ou controle de perdas.

Por que o código não inclui endpoint de produção?

Porque o livro é para iniciantes. Endpoint de produção aumenta risco. Quando você está aprendendo, é melhor remover caminhos perigosos.

Posso trocar BNBUSDT por BTCUSDT?

Pode. Defina:

```
export BOT_SYMBOL="BTCUSDT"
```

Depois rode price, klines e signal. Para ordens, confira filtros e quantidade.

Por que minha ordem de teste falha?

Possíveis motivos:

- chaves erradas;
- chave de produção usada na Testnet;
- quantidade inválida;

- símbolo sem saldo suficiente na Testnet;
- permissão de chave inadequada;
- instabilidade temporária.

Comece conferindo chaves e quantidade.

O que é melhor: Quantity ou QuoteOrderQty?

Depende da regra da ordem. Quantity informa quantidade do ativo base. QuoteOrderQty informa valor no ativo de cotação. Este projeto usa Quantity para simplificar.

Por que usar médias móveis se elas são limitadas?

Porque são fáceis de entender. Uma primeira estratégia deve ensinar estrutura, não impressionar. Depois você pode trocar a estratégia.

O robô deveria comprar automaticamente quando signal der BUY?

Não neste projeto. Separar sinal de execução reduz risco. Primeiro veja o sinal. Depois, se quiser, execute um comando explícito.

Posso rodar em loop a cada minuto?

Pode como exercício futuro, mas faça com cuidado. Um loop mal escrito pode enviar ordens repetidas. Antes de loop, implemente estado, logs e limites.

Preciso de banco de dados?

Para este projeto, não. Para um robô real, provavelmente sim. Você vai querer salvar ordens, sinais, erros e decisões.

Por que o projeto usa float64?

Para simplificar aprendizado. Em sistemas financeiros robustos, você deve estudar tipos decimais, arredondamento e filtros de precisão.

O Kindle aceita EPUB?

Sim, o KDP aceita EPUB quando segue as diretrizes Kindle. Ainda assim, use Kindle Previewer antes de publicar.

É melhor EPUB ou KPF?

Para publicação Kindle, a Amazon recomenda KPF como formato preferido quando criado pelo Kindle Create. Neste pacote, EPUB é o fluxo principal porque nasce direto do Markdown, e DOCX fica disponível para importar no Kindle Create.

Posso usar logo da Binance ou do Go na capa?

Evite, a menos que você tenha certeza sobre as regras de marca. Para teste KDP, uma capa tipográfica sem logos é mais segura.

Preciso informar uso de IA no KDP?

Se texto, imagem ou tradução forem gerados por ferramenta de IA, o KDP exige disclosure de conteúdo gerado por IA.

Se você apenas usou IA como assistência e revisou/criou substancialmente o conteúdo, a política diferencia AI-assisted de AI-generated. Na dúvida, seja transparente.

O livro promete resultado financeiro?

Não. E não deve prometer. A proposta é aprender programação, API e publicação de ebook técnico.

Como aumento a qualidade do livro antes de publicar?

Faça uma revisão humana. Rode os comandos. Tire prints próprios se quiser incluir imagens futuras. Peça para uma pessoa iniciante ler e marcar pontos confusos.

Como aumento a qualidade do código?

Adicione logs estruturados, testes de integração opcionais, tratamento de filtros de símbolo, arquivo de configuração, persistência local e uma camada de simulação.

O que estudar depois?

Boas próximas etapas:

- WebSocket Streams;
- backtesting;
- gerenciamento de risco;
- ordem limitada;
- logs;
- deploy;

- observabilidade;
- segurança de chaves.

Qual é a maior lição do projeto?

Separar partes. Configuração é uma parte. API é outra. Estratégia é outra. CLI é outra. Testes são outra. Quando cada parte tem um papel claro, o projeto fica menos assustador.

* * *

Apêndice E: Referências

- Go Release History: <https://go.dev/doc/devel/release>
- Go 1.26 Release Notes: <https://go.dev/doc/go1.26>
- Binance Spot API Docs: <https://developers.binance.com/docs/binance-spot-api-docs/README>
- Binance Go Connector:
<https://github.com/binance/binance-connector-go>
- KDP: formatos aceitos para eBooks:
https://kdp.amazon.com/pt_BR/help/topic/G200634390
- Kindle Create: https://kdp.amazon.com/en_US/help/topic/GUGQ4WDZ92F733GC
- Diretrizes de conteúdo KDP: <https://kdp.amazon.com/self-publishing/help?topicId=A2AH1EAPH0YKI9>

* * *

Encerramento

Se você chegou até aqui, completou uma jornada que parece pequena, mas não é: você organizou um projeto Go, usou uma API real, separou configuração, escreveu uma estratégia simples, adicionou travas de segurança, rodou testes e preparou um livro para publicação.

Esse é o ponto. Não é sobre sair com um robô milagroso. É sobre sair com um processo.

O próximo passo natural é escolher uma melhoria por vez:

- adicionar logs estruturados;
- salvar resultados em arquivo;
- permitir intervalos diferentes;
- criar backtest simples;
- adicionar WebSocket Streams;
- melhorar a gestão de risco;
- revisar o livro com leitores reais.

Um bom projeto cresce melhor quando nasce pequeno e compreensível.